LearnHub

MERN Stack Learning Management System

Final Project Report — MERN Stack Web Development

Question 1	MERN LMS: React pages, Express REST APIs, MongoDB
Question 2	JWT auth, role-based access, enrollment, deployment readiness
Stack	React (Vite), React Router, Axios, React Bootstrap, Node.js, Express, Mongoose, MongoDB, JWT, bcrypt
Date	Tue Sep 29 2026

1. Project Overview

LearnHub is a Learning Management System where **students** browse and enroll in courses and track progress, **instructors** create, edit and delete courses and upload lessons, and **admins** manage users and courses and view analytics. The React frontend talks to the Express REST API through Axios; data is stored in MongoDB via Mongoose. Authentication uses JWT with bcrypt-hashed passwords; secrets live in a `.env` file.

Roles and permissions

Role	Can do
Student	Browse courses, enroll, view enrolled courses, mark lessons complete (progress)
Instructor	Create / edit / delete own courses, upload lessons
Admin	Manage all users (change role, delete) and courses, view analytics

2. Project Structure

project tree (excluding node_modules, dist)

```
./.gitignore
./LearnHub-Final-Report.pdf
./README.md
./backend/.env
./backend/.env.example
./backend/.gitignore
./backend/config/db.js
./backend/controllers/adminController.js
./backend/controllers/authController.js
./backend/controllers/courseController.js
./backend/controllers/enrollmentController.js
./backend/middleware/auth.js
./backend/middleware/error.js
./backend/models/Course.js
./backend/models/Enrollment.js
./backend/models/User.js
./backend/package.json
./backend/routes/adminRoutes.js
./backend/routes/authRoutes.js
./backend/routes/courseRoutes.js
./backend/routes/enrollmentRoutes.js
./backend/seed.js
./backend/server.js
./backend/tests/api.test.js
./frontend/.env.example
./frontend/.gitignore
./frontend/index.html
./frontend/package.json
./frontend/src/App.jsx
./frontend/src/components/CourseCard.jsx
./frontend/src/components/Footer.jsx
./frontend/src/components/Loader.jsx
./frontend/src/components/NavBar.jsx
./frontend/src/components/ProtectedRoute.jsx
./frontend/src/context/AuthContext.jsx
./frontend/src/index.css
./frontend/src/main.jsx
./frontend/src/pages/AdminDashboard.jsx
./frontend/src/pages/CourseDetail.jsx
./frontend/src/pages/Courses.jsx
./frontend/src/pages/Home.jsx
./frontend/src/pages/InstructorDashboard.jsx
./frontend/src/pages/Login.jsx
./frontend/src/pages/NotFound.jsx
./frontend/src/pages/Register.jsx
./frontend/src/pages/StudentDashboard.jsx
./frontend/src/services/api.js
./frontend/vite.config.js
```

3. Environment and Servers Running

● ● ● MongoDB Windows service

```
Name     Status StartType
----     ------ ---------
MongoDB Running Automatic
```

● ● ● backend: node server.js

```
◇ injected env (6) from .env
MongoDB connected: 127.0.0.1/learnhub
LearnHub API running on http://localhost:5000
```

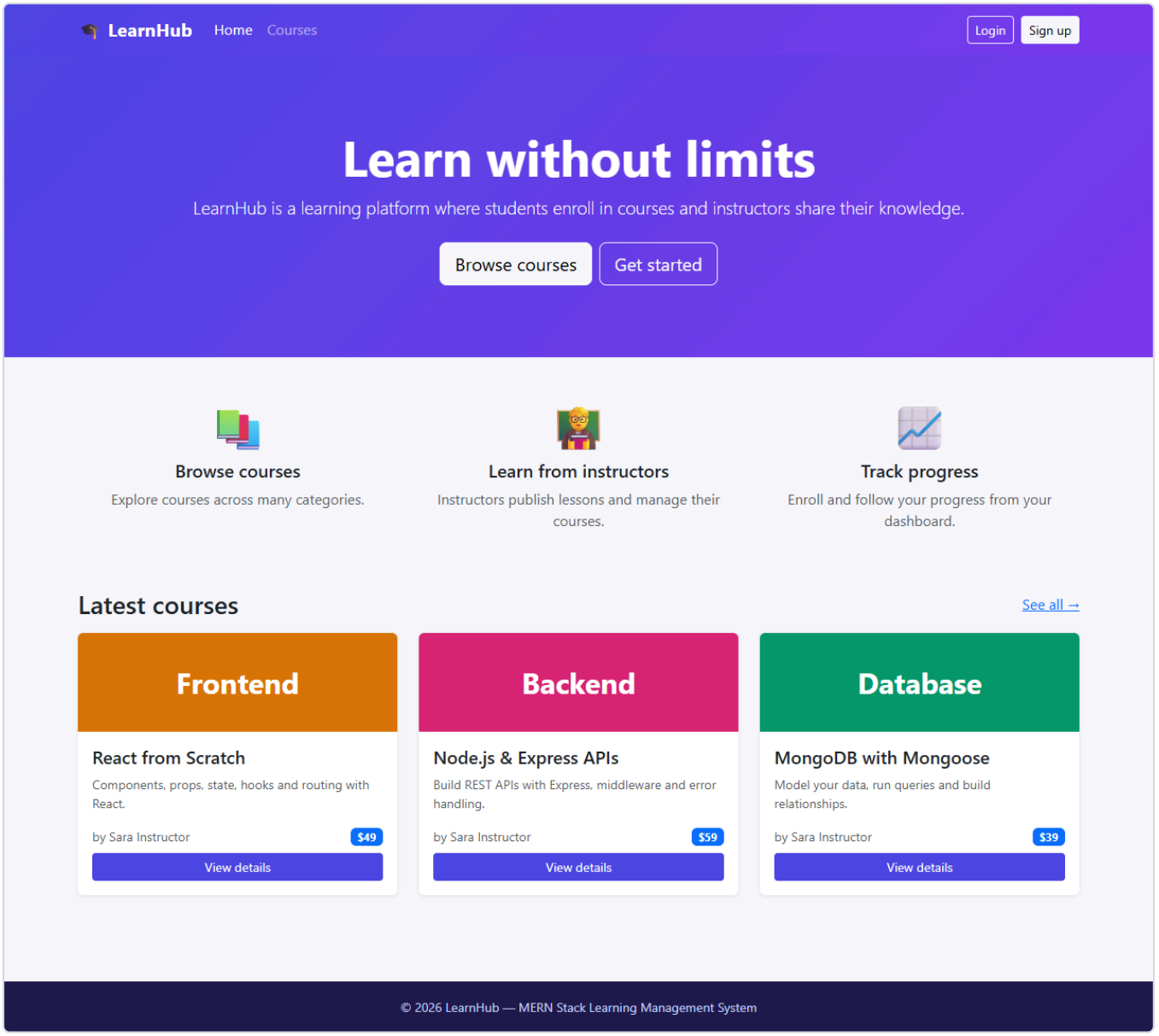
● ● ● frontend: npm run build (production build)

```
> learnhub-frontend@1.0.0 build
> vite build

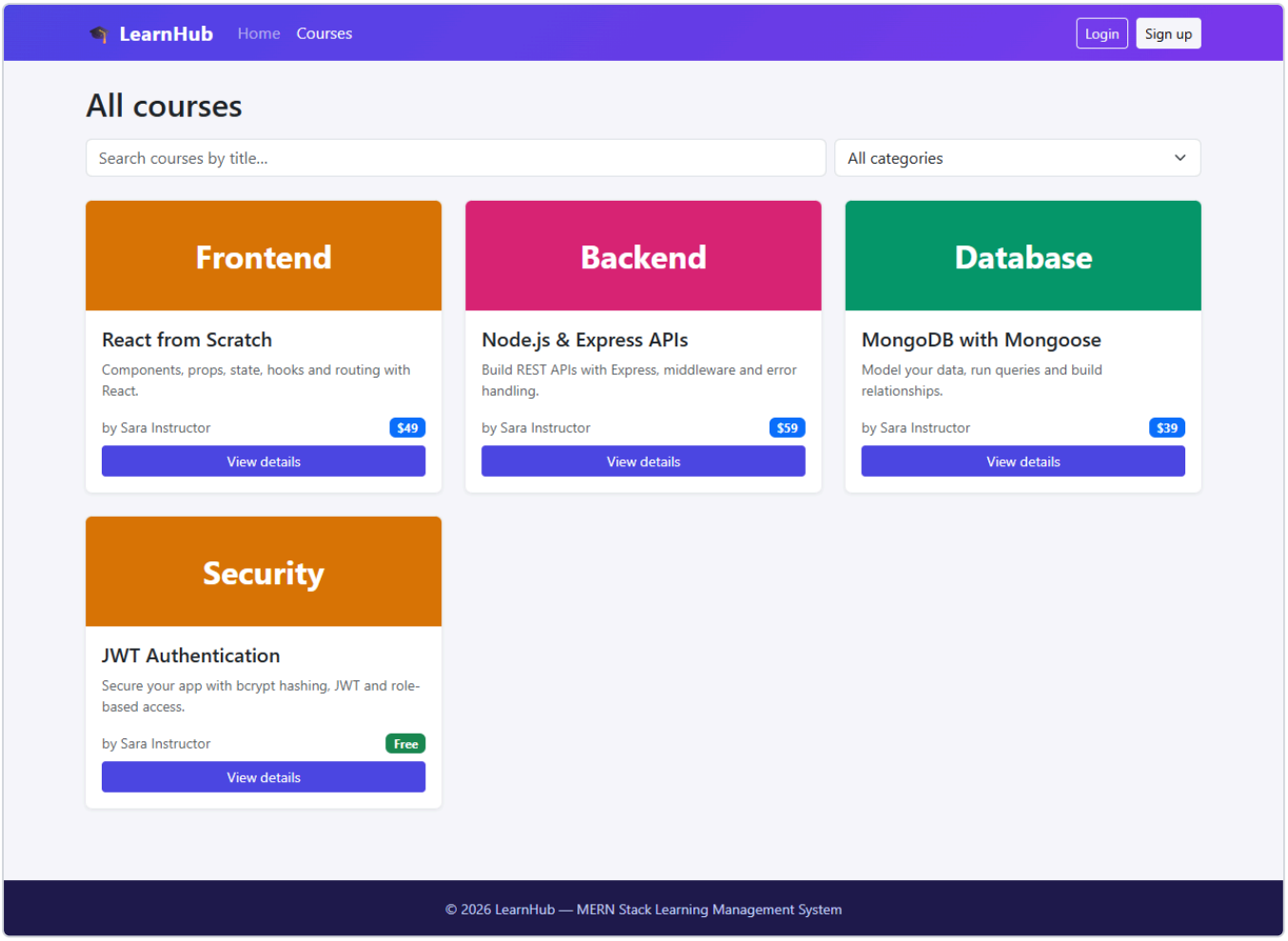
vite v8.3.1 building client environment for production...
transforming...
✓ 403 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.42 kB │ gzip:   0.28 kB
dist/assets/index-COzP_-Fc.css  231.07 kB │ gzip:  31.14 kB
dist/assets/index-B2oK8b74.js   389.66 kB │ gzip: 123.66 kB

✓ built in 197ms
```

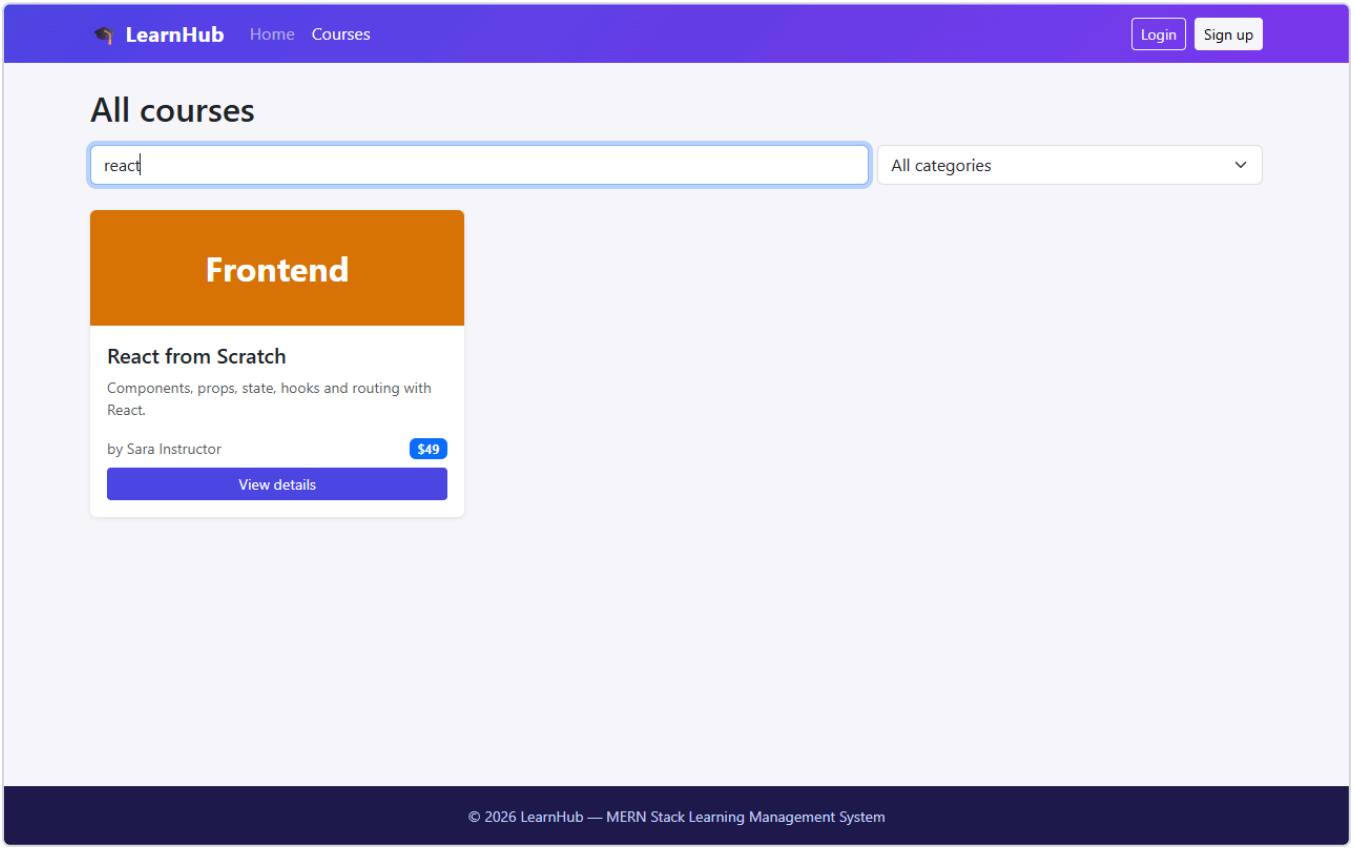
4. Question 1 — Frontend Pages



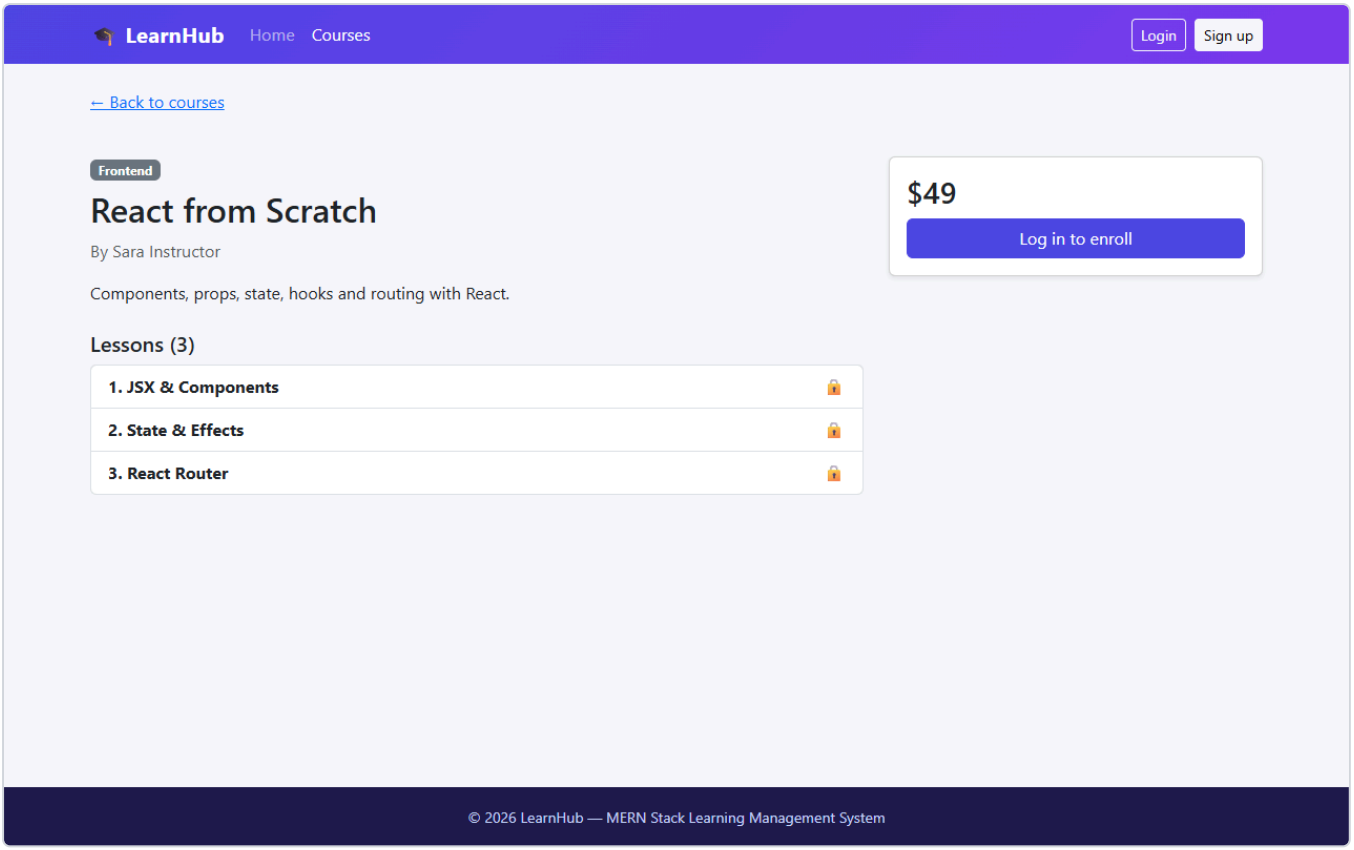
Home page — hero, features and latest courses fetched from the API



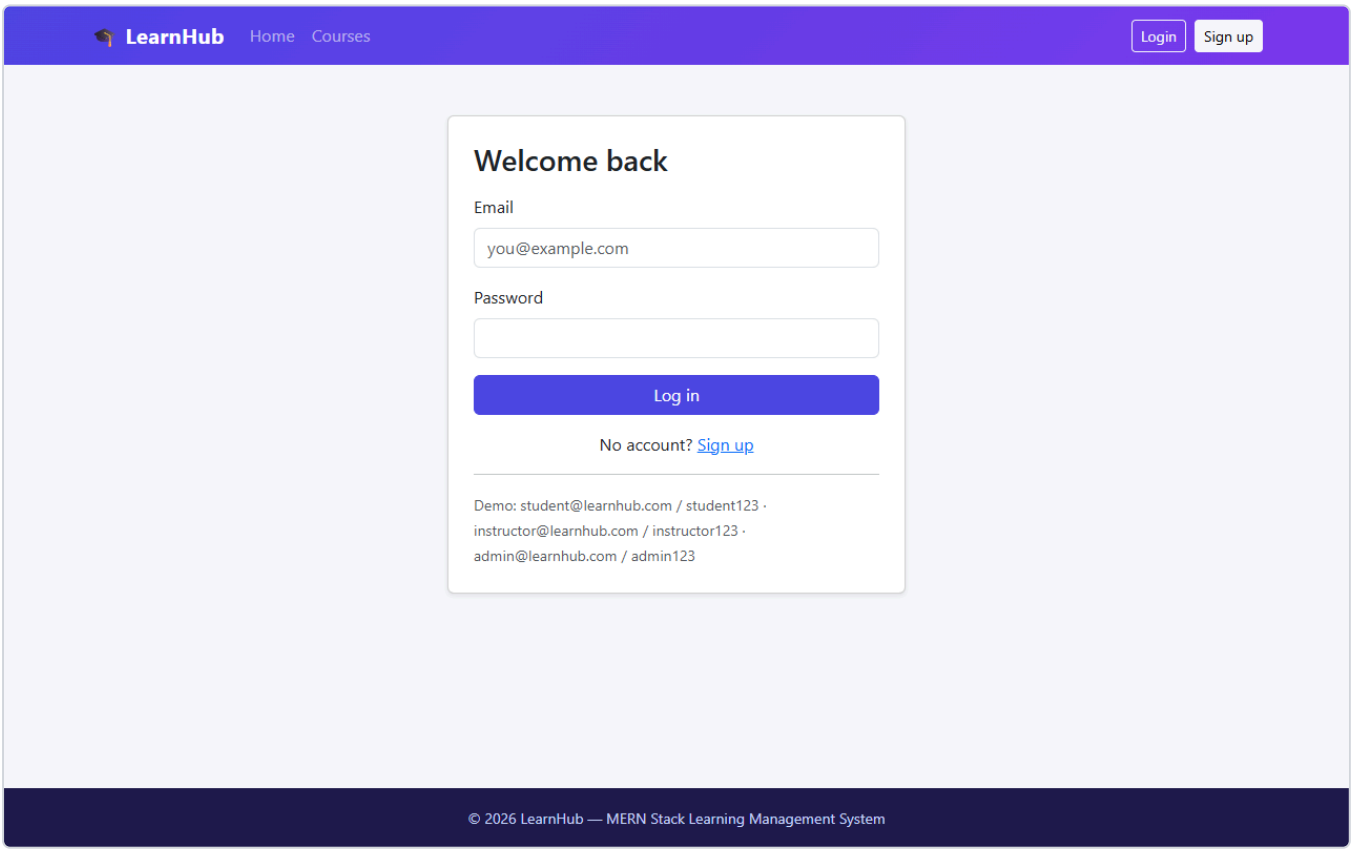
Course Listing page — all courses from GET /api/courses



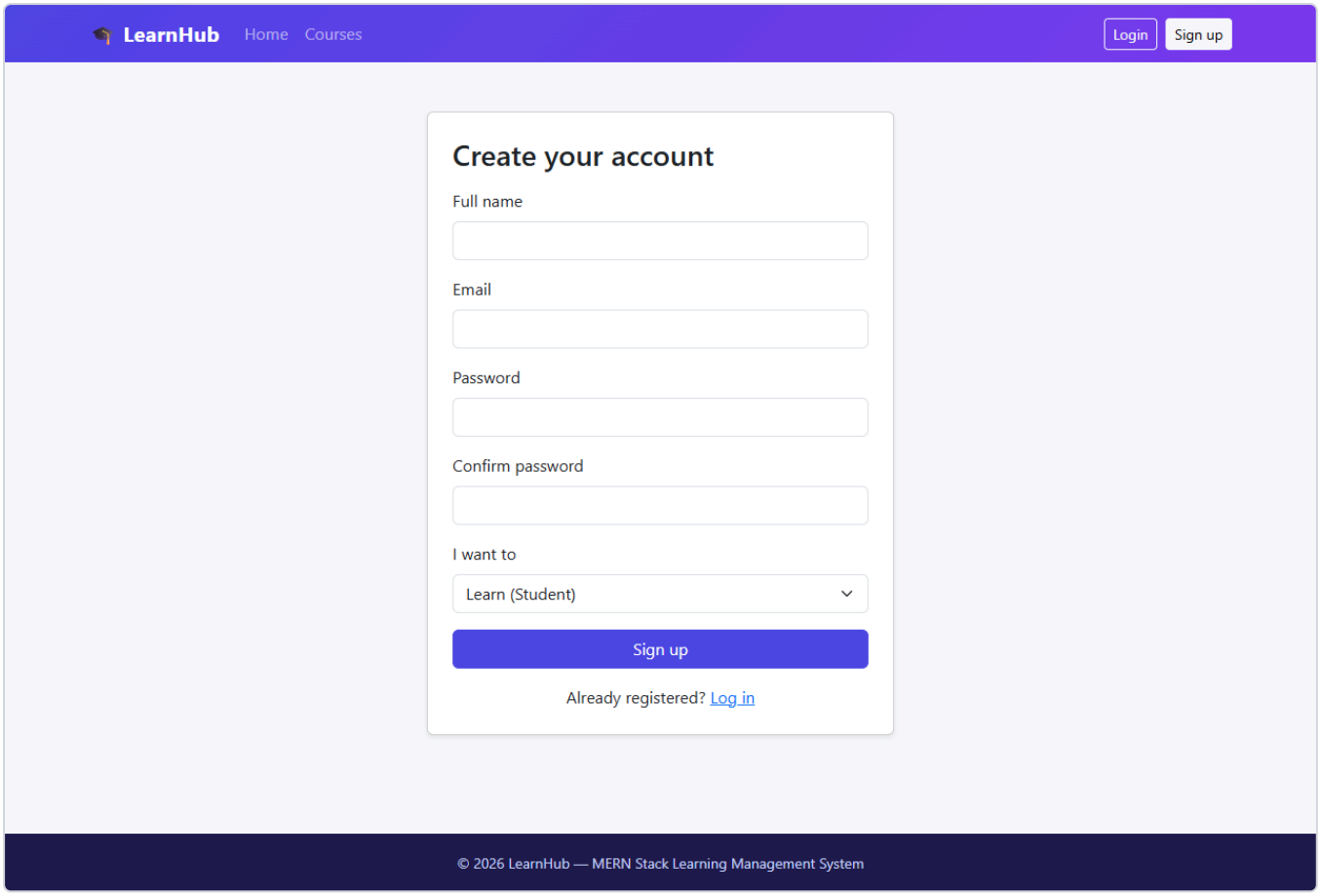
Course search — list re-fetches from the backend as the user types



Course Detail page — visitors see a "Log in to enroll" button and locked lessons



Login page



Register page

5. Question 1 — Backend REST API Evidence

The following requests were sent to the running server (Postman-style capture; JWTs shortened). Status codes show both successful flows and validation/authorization failures.

POST /api/register — Register a student (valid) **201**

REQUEST BODY

```
{"name":"Api Tester","email":"apitest1790694039059@example.com","password":"secret123"}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJIUzI1NiIs…",
  "user": {
    "_id": "6abbd297fa685c3f9ed7aad5",
    "name": "Api Tester",
    "email": "apitest1790694039059@example.com",
    "role": "student"
  }
}
```

POST /api/register — Register with invalid email **400**

REQUEST BODY

```
{"name":"Bad","email":"nope","password":"secret123"}
```

RESPONSE

```
{
  "message": "Please provide a valid email"
}
```

POST /api/register — Register duplicate email **409**

REQUEST BODY

```
{"name":"Api Tester","email":"apitest1790694039059@example.com","password":"secret123"}
```

RESPONSE

```
{
  "message": "email already exists"
}
```

POST /api/login — Login as instructor **200**

REQUEST BODY

```
{"email":"instructor@learnhub.com","password":"instructor123"}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJIUzI1NiIs…",
  "user": {
    "_id": "6abbd292c3a6842704560ec6",
    "name": "Sara Instructor",
    "email": "instructor@learnhub.com",
    "role": "instructor"
  }
}
```

POST /api/login — Login with wrong password **401**

REQUEST BODY

```
{"email":"instructor@learnhub.com","password":"wrong"}
```

RESPONSE

```
{
  "message": "Invalid email or password"
}
```

POST /api/login — Login as student **200**

REQUEST BODY

```
{"email":"student@learnhub.com","password":"student123"}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJIUzI1NiIs…",
  "user": {
    "_id": "6abbd292c3a6842704560ec7",
    "name": "Ali Student",
    "email": "student@learnhub.com",
    "role": "student"
  }
}
```

POST /api/login — Login as admin **200**

REQUEST BODY

```
{"email":"admin@learnhub.com","password":"admin123"}
```

RESPONSE

```
{
  "token": "eyJhbGciOiJIUzI1NiIs…",
  "user": {
    "_id": "6abbd292c3a6842704560ec5",
    "name": "Admin User",
    "email": "admin@learnhub.com",
    "role": "admin"
  }
}
```

GET /api/courses?search=react — List all courses (public) **200**

RESPONSE


```
[
  {
    "_id": "6abbd292c3a6842704560ec8",
    "title": "React from Scratch",
    "description": "Components, props, state, hooks and routing with React.",
    "instructor": {
      "_id": "6abbd292c3a6842704560ec6",
      "name": "Sara Instructor",
      "email": "instructor@learnhub.com"
    },
    "category": "Frontend",
    "price": 49,
    "lessons": [
      {
        "title": "JSX & Components",
        "content": "Building blocks of React.",
        "videoUrl": "",
        "_id": "6abbd292c3a6842704560ec9",
        "createdAt": "2026-09-29T15:00:34.343Z",
        "updatedAt": "2026-09-29T15:00:34.343Z"
      },
      {
        "title": "State & Effects",
        "content": "useState and useEffect.",
        "videoUrl": "",
        "_id": "6abbd292c3a6842704560eca",
        "createdAt": "2026-09-29T15:00:34.343Z",
        "updatedAt": "2026-09-29T15:00:34.343Z"
      },
      {

  ... (truncated)
```


POST /api/courses — Create course (instructor) **201**

REQUEST BODY


```
{"title":"Postman Demo Course","description":"Created via API","category":"Backend","price":20}
```


HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
{
  "title": "Postman Demo Course",
  "description": "Created via API",
  "instructor": {
    "_id": "6abbd292c3a6842704560ec6",
    "name": "Sara Instructor",
    "email": "instructor@learnhub.com"
  },
  "category": "Backend",
  "price": 20,
  "_id": "6abbd297fa685c3f9ed7aadb",
  "lessons": [],
  "createdAt": "2026-09-29T15:00:39.396Z",
  "updatedAt": "2026-09-29T15:00:39.396Z",
  "__v": 0
}
```

POST /api/courses — Create course with missing fields **400**

REQUEST BODY

```
{"title":"Only title"}
```

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE

```
{
  "message": "Description is required, Category is required"
}
```

POST /api/courses — Create course as student (forbidden) **403**

REQUEST BODY

```
{"title":"x","description":"y","category":"z","price":1}
```

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE

```
{
  "message": "Role 'student' is not allowed to access this resource"
}
```

PUT /api/courses/6abbd297fa685c3f9ed7aadb — Update course (owner) **200**

REQUEST BODY

```
{"price":35}
```

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE

```
{
  "_id": "6abbd297fa685c3f9ed7aadb",
  "title": "Postman Demo Course",
  "description": "Created via API",
  "instructor": {
    "_id": "6abbd292c3a6842704560ec6",
    "name": "Sara Instructor",
    "email": "instructor@learnhub.com"
  },
  "category": "Backend",
  "price": 35,
  "lessons": [],
  "createdAt": "2026-09-29T15:00:39.396Z",
  "updatedAt": "2026-09-29T15:00:39.402Z",
  "__v": 0
}
```

GET /api/my-courses — Get my courses without token **401**

RESPONSE

```
{
  "message": "Not authorized, no token provided"
}
```

POST /api/enroll — Enroll student in course

201

REQUEST BODY

```
{"courseId":"6abbd297fa685c3f9ed7aadb"}
```

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
{
  "student": "6abbd292c3a6842704560ec7",
  "course": {
    "_id": "6abbd297fa685c3f9ed7aadb",
    "title": "Postman Demo Course",
    "description": "Created via API",
    "instructor": "6abbd292c3a6842704560ec6",
    "category": "Backend",
    "price": 35,
    "lessons": [],
    "createdAt": "2026-09-29T15:00:39.396Z",
    "updatedAt": "2026-09-29T15:00:39.402Z",
    "__v": 0
  },
  "progress": 0,
  "completedLessons": [],
  "_id": "6abbd297fa685c3f9ed7aadd",
  "createdAt": "2026-09-29T15:00:39.406Z",
  "updatedAt": "2026-09-29T15:00:39.406Z",
  "__v": 0
}
```


POST /api/enroll — Enroll again (duplicate)

409

REQUEST BODY

```
{"courseId":"6abbd297fa685c3f9ed7aadb"}
```

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE

```
{
  "message": "You are already enrolled in this course"
}
```

GET /api/my-courses — Get enrolled courses

200

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
[
  {
    "_id": "6abbd297fa685c3f9ed7aadd",
    "student": "6abbd292c3a6842704560ec7",
    "course": {
      "_id": "6abbd297fa685c3f9ed7aadb",
      "title": "Postman Demo Course",
      "description": "Created via API",
      "instructor": {
        "_id": "6abbd292c3a6842704560ec6",
        "name": "Sara Instructor"
      },
      "category": "Backend",
      "price": 35,
      "lessons": [],
      "createdAt": "2026-09-29T15:00:39.396Z",
      "updatedAt": "2026-09-29T15:00:39.402Z",
      "__v": 0
    },
    "progress": 0,
    "completedLessons": [],
    "createdAt": "2026-09-29T15:00:39.406Z",
    "updatedAt": "2026-09-29T15:00:39.406Z",
    "__v": 0
  },
  {
    "_id": "6abbd292c3a6842704560ed2",
    "student": "6abbd292c3a6842704560ec7",
    "course": {
      "_id": "6abbd292c3a6842704560ec8",
      "title": "React from Scratch",
      "description": "Components, props, state, h
  ... (truncated)
```

GET /api/admin/analytics — Admin analytics

200

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
{
  "users": 4,
  "students": 2,
  "instructors": 1,
  "admins": 1,
  "courses": 5,
  "enrollments": 2,
  "popular": [
    {
      "enrollments": 1,
      "title": "React from Scratch"
    },
    {
      "enrollments": 1,
      "title": "Postman Demo Course"
    }
  ],
  "byCategory": [
    {
      "_id": "Backend",
      "count": 2
    },
    {
      "_id": "Database",
      "count": 1
    },
    {
      "_id": "Frontend",
      "count": 1
    },
    {
      "_id": "Security",
      "count": 1
    }
  ]
}
```


GET /api/admin/analytics — Analytics as instructor (forbidden)

403

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
{
  "message": "Role 'instructor' is not allowed to access this resource"
}
```


DELETE /api/courses/6abbd297fa685c3f9ed7aadb — Delete course (owner)

200

HEADER: AUTHORIZATION: BEARER <JWT>

RESPONSE


```
{
  "message": "Course deleted"
}
```

6. Important Code Sections

Mongoose models

backend/models/User.js (lines 1-27)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: [true, 'Name is required'], trim: true, minlength: 2 },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email'],
    },
    password: { type: String, required: [true, 'Password is required'], minlength: 6, select: false },
    role: { type: String, enum: ['admin', 'instructor', 'student'], default: 'student' },
  },
  { timestamps: true }
);

userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, 10);
});

userSchema.methods.matchPassword = function (plain) {
  return bcrypt.compare(plain, this.password);
```

backend/models/Course.js (lines 1-25)

```
const mongoose = require('mongoose');

const lessonSchema = new mongoose.Schema(
  {
    title: { type: String, required: [true, 'Lesson title is required'], trim: true },
    content: { type: String, default: '' },
    videoUrl: { type: String, default: '' },
  },
  { timestamps: true }
);

const courseSchema = new mongoose.Schema(
  {
    title: { type: String, required: [true, 'Title is required'], trim: true },
    description: { type: String, required: [true, 'Description is required'] },
    instructor: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    category: { type: String, required: [true, 'Category is required'], trim: true },
    price: { type: Number, required: [true, 'Price is required'], min: [0, 'Price cannot be negative'],
default: 0 },
    lessons: [lessonSchema],
  },
  { timestamps: true }
);

module.exports = mongoose.model('Course', courseSchema);
```

backend/models/Enrollment.js (lines 1-15)

```
const mongoose = require('mongoose');

const enrollmentSchema = new mongoose.Schema(
  {
    student: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    course: { type: mongoose.Schema.Types.ObjectId, ref: 'Course', required: true },
```

```
    progress: { type: Number, min: 0, max: 100, default: 0 },
    completedLessons: [{ type: mongoose.Schema.Types.ObjectId }],
  },
  { timestamps: true }
);

enrollmentSchema.index({ student: 1, course: 1 }, { unique: true });

module.exports = mongoose.model('Enrollment', enrollmentSchema);
```

JWT authentication and role middleware

backend/middleware/auth.js (lines 1-31)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Verifies the Bearer token and attaches the user to req.user
const protect = async (req, res, next) => {
  try {
    const header = req.headers.authorization || '';
    if (!header.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Not authorized, no token provided' });
    }
    const decoded = jwt.verify(header.split(' ')[1], process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);
    if (!user) return res.status(401).json({ message: 'User no longer exists' });
    req.user = user;
    next();
  } catch (err) {
    const message = err.name === 'TokenExpiredError' ? 'Session expired, please log in again' : 'Invalid
token';
    res.status(401).json({ message });
  }
};

// Restricts a route to the given roles
const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: `Role '${req.user.role}' is not allowed to access this resource`
});
  }
  next();
};

module.exports = { protect, authorize };
```

backend/controllers/authController.js (lines 1-47)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const signToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN ||
'7d' });

const respond = (res, status, user) =>
  res.status(status).json({
    token: signToken(user._id),
    user: { _id: user._id, name: user.name, email: user.email, role: user.role },
  });

// POST /api/register  -- public signup can only create student or instructor accounts
exports.register = async (req, res, next) => {
  try {
    const { name, email, password, role } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ message: 'Name, email and password are required' });
    }
    if (password.length < 6) return res.status(400).json({ message: 'Password must be at least 6 characters'
});
    if (role && !['student', 'instructor'].includes(role)) {
      return res.status(400).json({ message: "Role must be 'student' or 'instructor'" });
    }
    const user = await User.create({ name, email, password, role: role || 'student' });
```

```
    respond(res, 201, user);
  } catch (err) {
    next(err);
  }
};

// POST /api/login
exports.login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) return res.status(400).json({ message: 'Email and password are required' });
    const user = await User.findOne({ email: String(email).toLowerCase() }).select('+password');
    if (!user || !(await user.matchPassword(password))) {
      return res.status(401).json({ message: 'Invalid email or password' });
    }
    respond(res, 200, user);
  } catch (err) {
    next(err);
  }
};

// GET /api/me
exports.me = (req, res) => res.json(req.user);
```

Enrollment endpoints

backend/controllers/enrollmentController.js (lines 1-32)

```
const Course = require('../models/Course');
const Enrollment = require('../models/Enrollment');

// POST /api/enroll  { courseId }  (student)
exports.enroll = async (req, res, next) => {
  try {
    const { courseId } = req.body;
    if (!courseId) return res.status(400).json({ message: 'courseId is required' });
    const course = await Course.findById(courseId);
    if (!course) return res.status(404).json({ message: 'Course not found' });
    if (await Enrollment.findOne({ student: req.user._id, course: courseId })) {
      return res.status(409).json({ message: 'You are already enrolled in this course' });
    }
    const enrollment = await Enrollment.create({ student: req.user._id, course: courseId });
    res.status(201).json(await enrollment.populate('course'));
  } catch (err) {
    next(err);
  }
};

// GET /api/my-courses  (student)
exports.myCourses = async (req, res, next) => {
  try {
    const enrollments = await Enrollment.find({ student: req.user._id })
      .populate({ path: 'course', populate: { path: 'instructor', select: 'name' } })
      .sort('-createdAt');
    res.json(enrollments.filter((e) => e.course));
  } catch (err) {
    next(err);
  }
};
```

Route protection

backend/routes/courseRoutes.js (lines 1-15)

```
const router = require('express').Router();
const c = require('../controllers/courseController');
const { protect, authorize } = require('../middleware/auth');

router.get('/', c.getCourses);
router.get('/mine', protect, authorize('instructor', 'admin'), c.myCreatedCourses);
router.get('/:id', c.getCourse);
router.post('/', protect, authorize('instructor', 'admin'), c.createCourse);
```

```
router.put('/:id', protect, authorize('instructor', 'admin'), c.updateCourse);
router.delete('/:id', protect, authorize('instructor', 'admin'), c.deleteCourse);
router.post('/:id/lessons', protect, authorize('instructor', 'admin'), c.addLesson);
router.delete('/:id/lessons/:lessonId', protect, authorize('instructor', 'admin'), c.deleteLesson);

module.exports = router;
```

Frontend: protected route and Axios service

frontend/src/components/ProtectedRoute.jsx (lines 1-25)

```
import { Navigate, useLocation } from 'react-router-dom';
import { Alert, Container } from 'react-bootstrap';
import { useAuth, dashboardPath } from '../context/AuthContext';
import Loader from './Loader';

// Blocks a page unless the user is logged in (and has one of the allowed roles)
export default function ProtectedRoute({ roles, children }) {
  const { user, loading } = useAuth();
  const location = useLocation();

  if (loading) return <Loader />;
  if (!user) return <Navigate to="/login" state={{ from: location.pathname, message: 'Please log in to
continue.' }} replace />;
  if (roles && !roles.includes(user.role)) {
    return (
      <Container className="py-5">
        <Alert variant="danger">
          <Alert.Heading>Access denied</Alert.Heading>
          Your role (<strong>{user.role}</strong>) is not allowed to view this page.{' '}
          <Alert.Link href={dashboardPath(user.role)}>Go to your dashboard</Alert.Link>
        </Alert>
      </Container>
    );
  }
  return children;
}
```

frontend/src/services/api.js (lines 1-14)

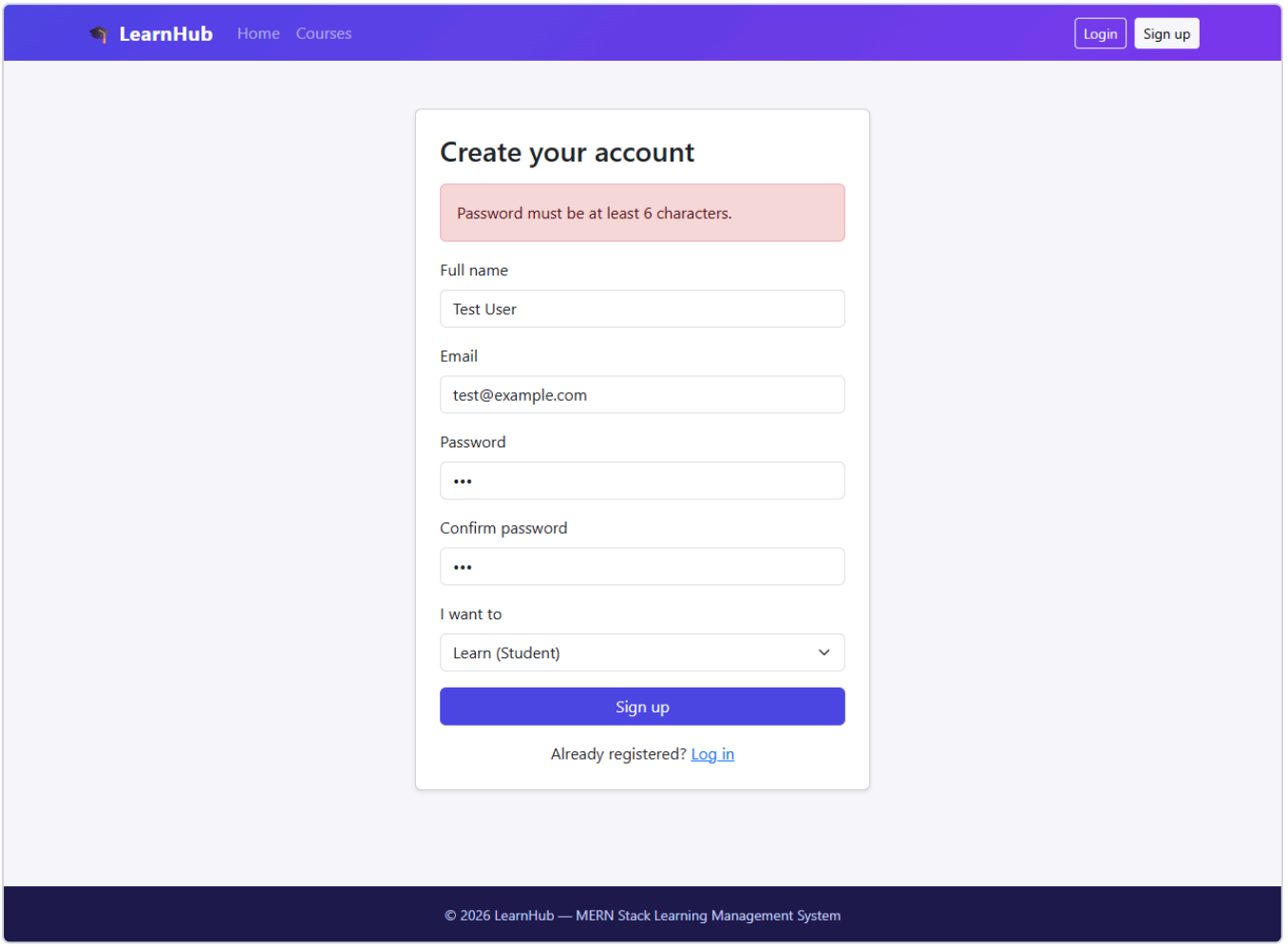
```
import axios from 'axios';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL || '/api' });

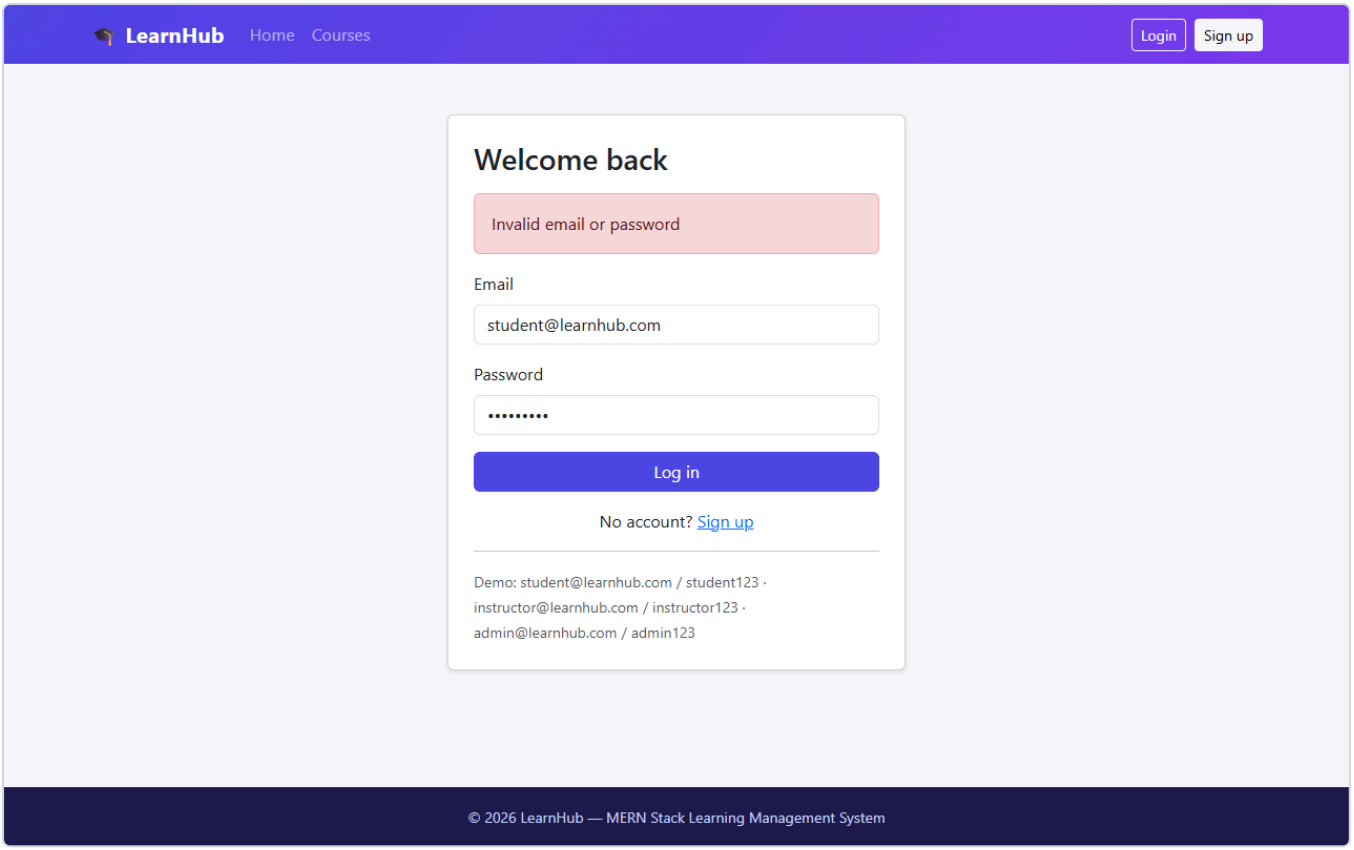
// Attach the JWT to every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Turn any failed request into a readable message
export const errorMessage = (err) =>
  err.response?.data?.message || (err.request ? 'Cannot reach the server. Is the backend running?' :
err.message);
```

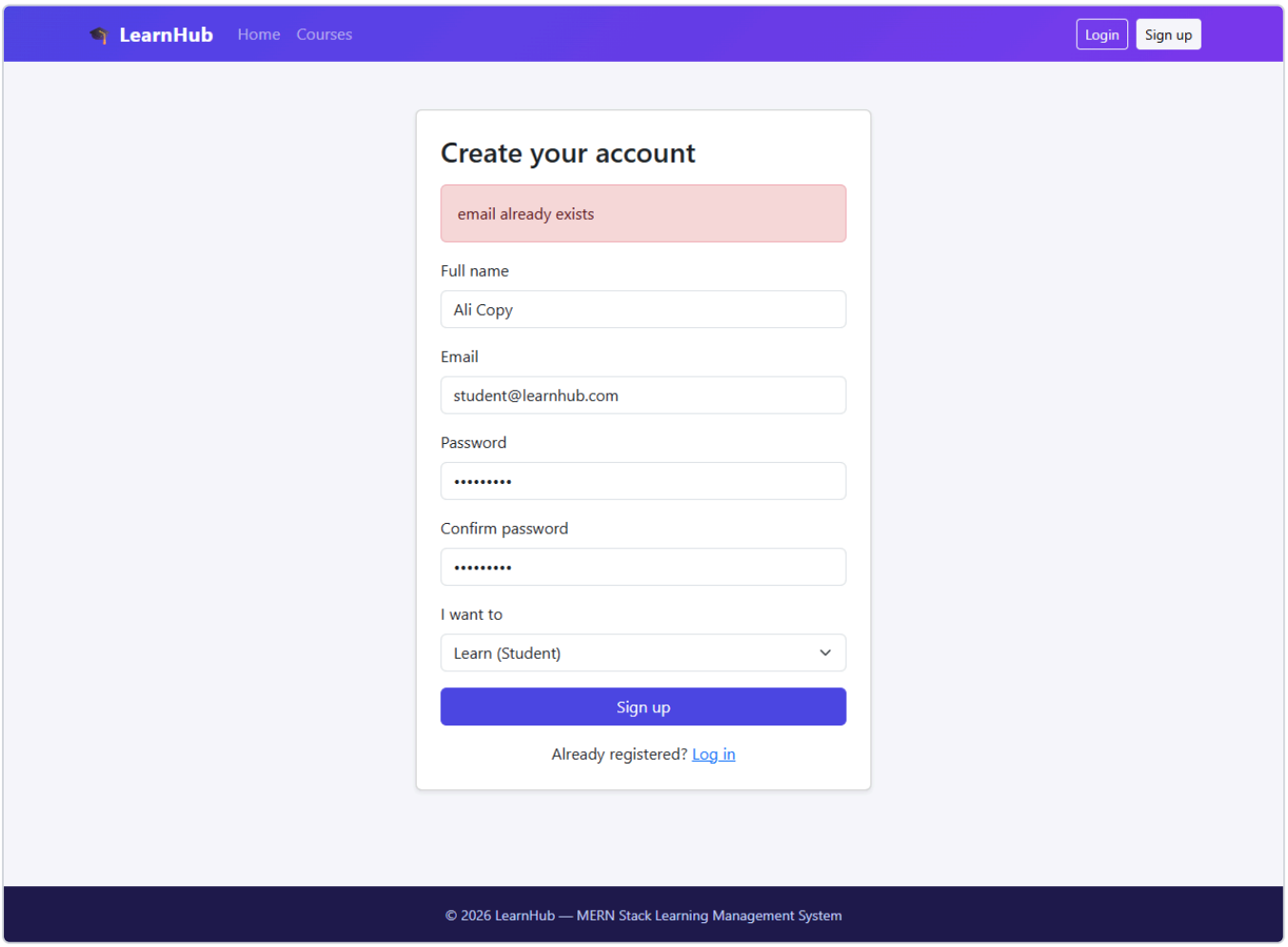
7. Question 2 — Error Handling and Validation



Client-side validation: password too short

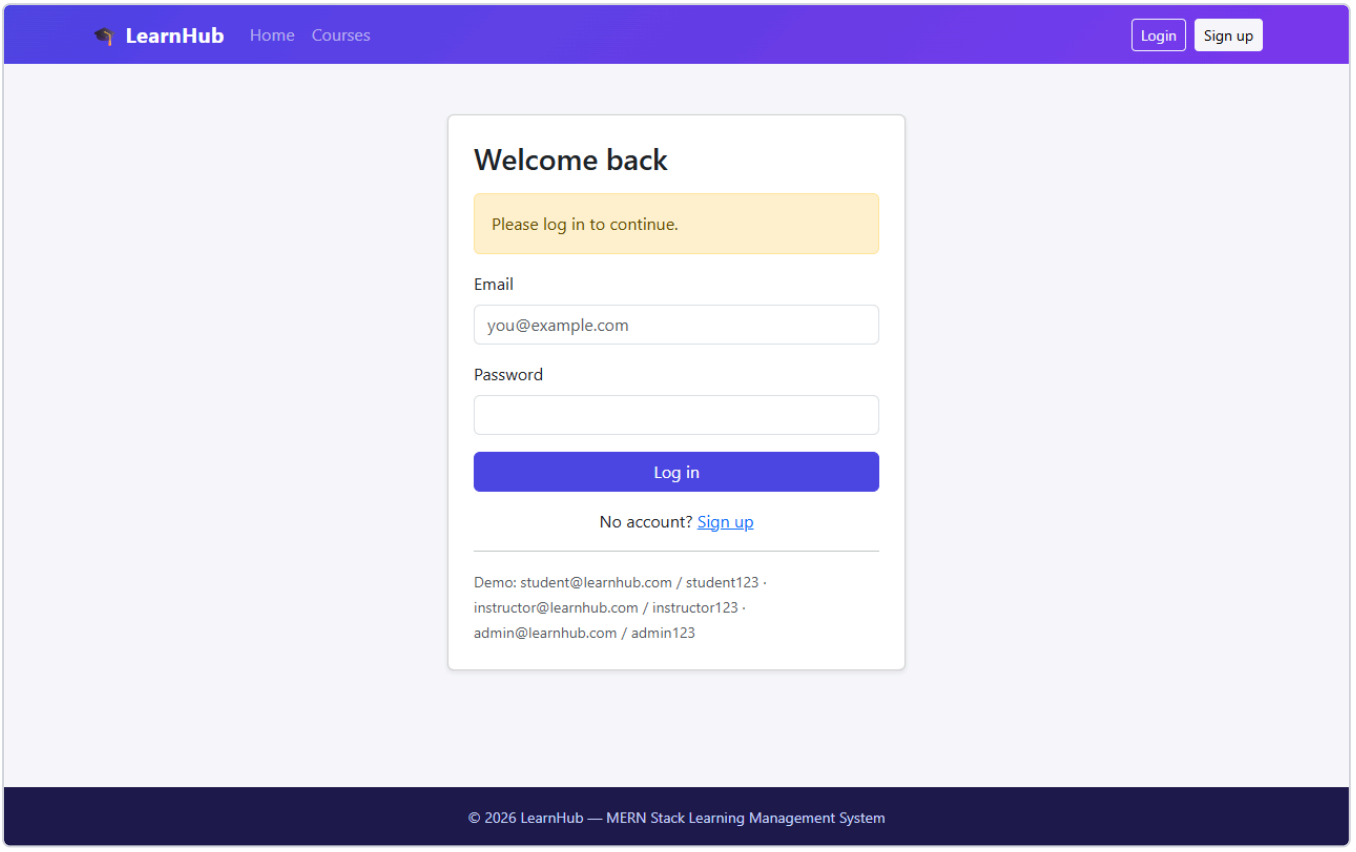


Server error surfaced in the UI: wrong password (401)

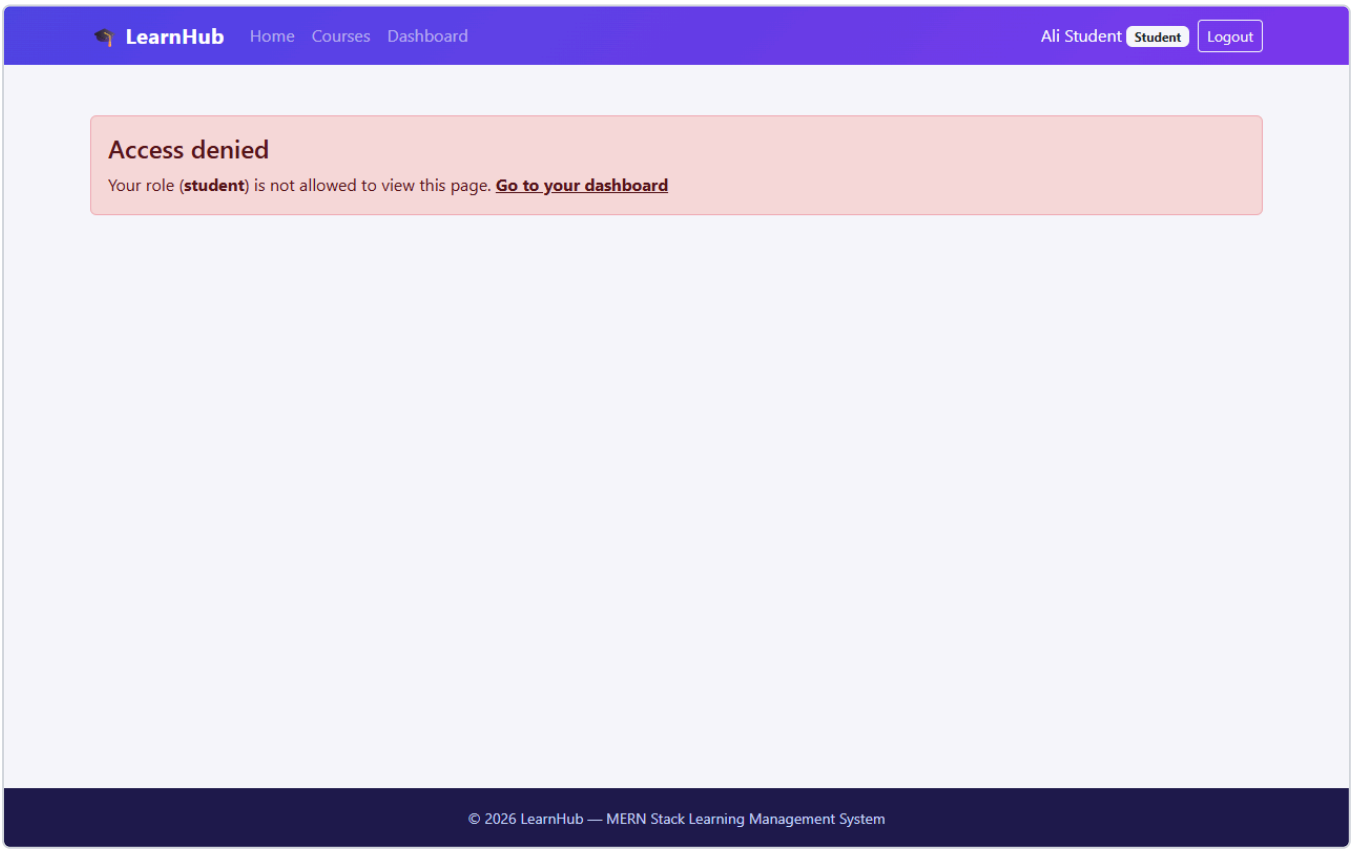


Duplicate email rejected by the API (409)

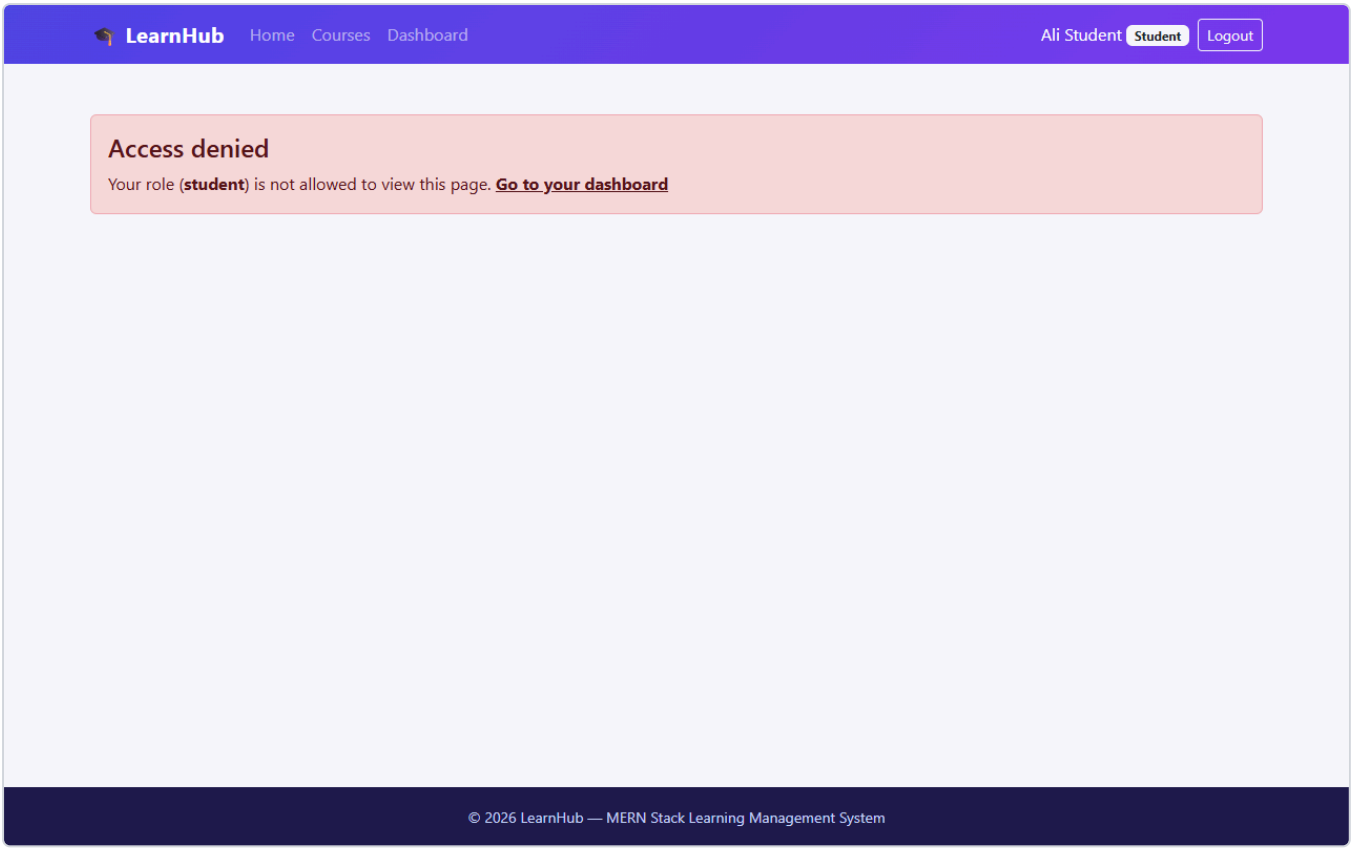
8. Question 2 — Protected Routes and Role-Based Dashboards



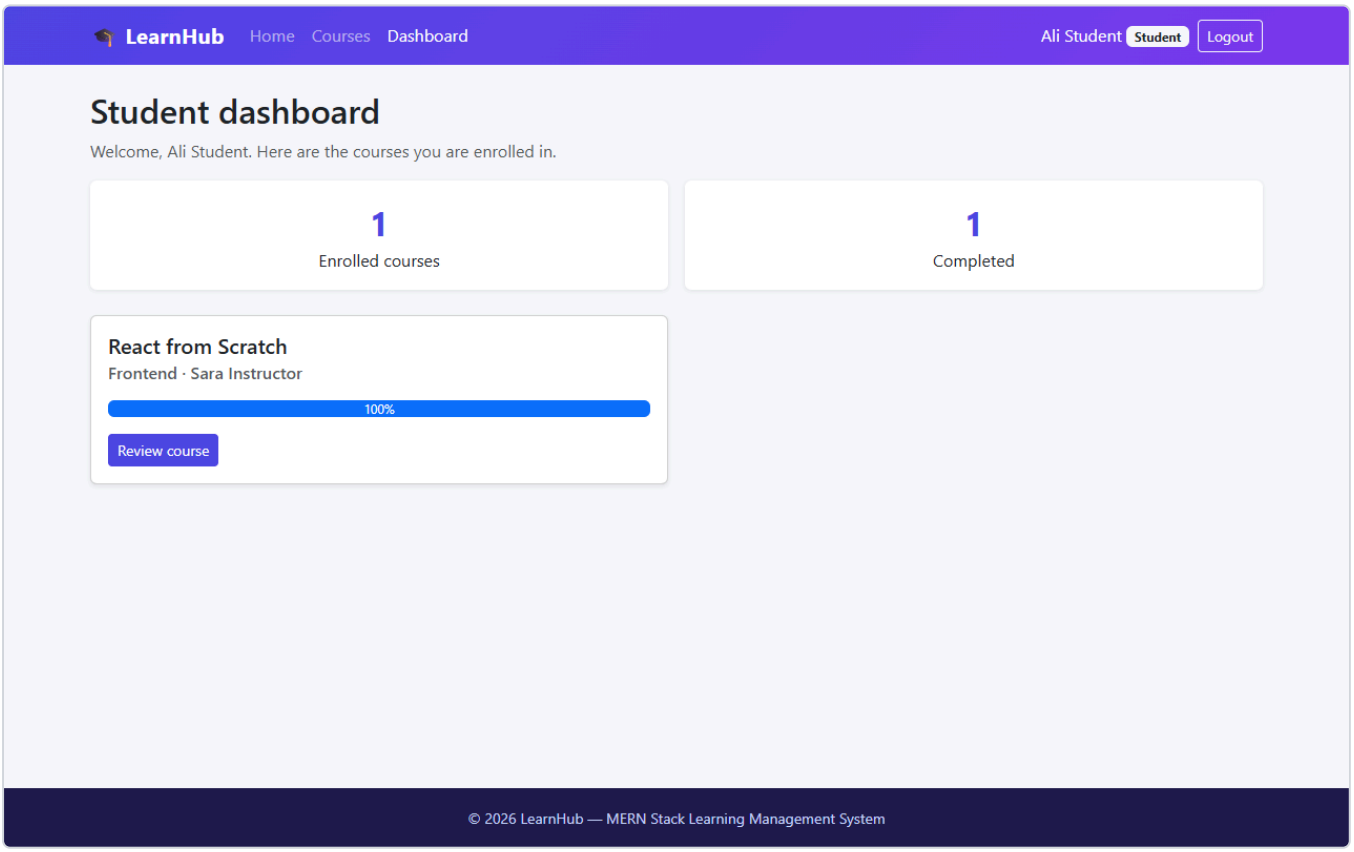
Visiting /dashboard/student without logging in redirects to Login with a message



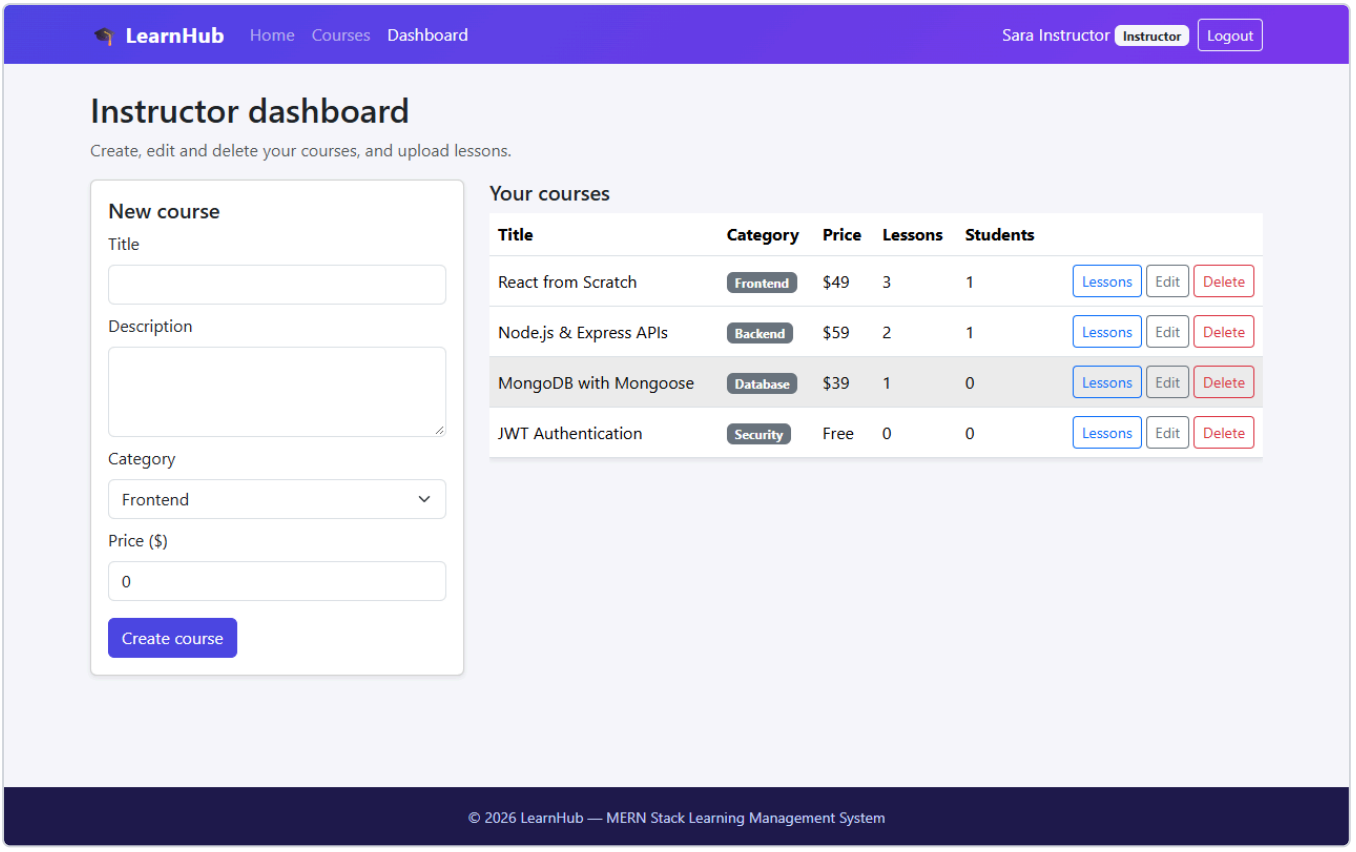
A student opening the admin dashboard gets "Access denied"



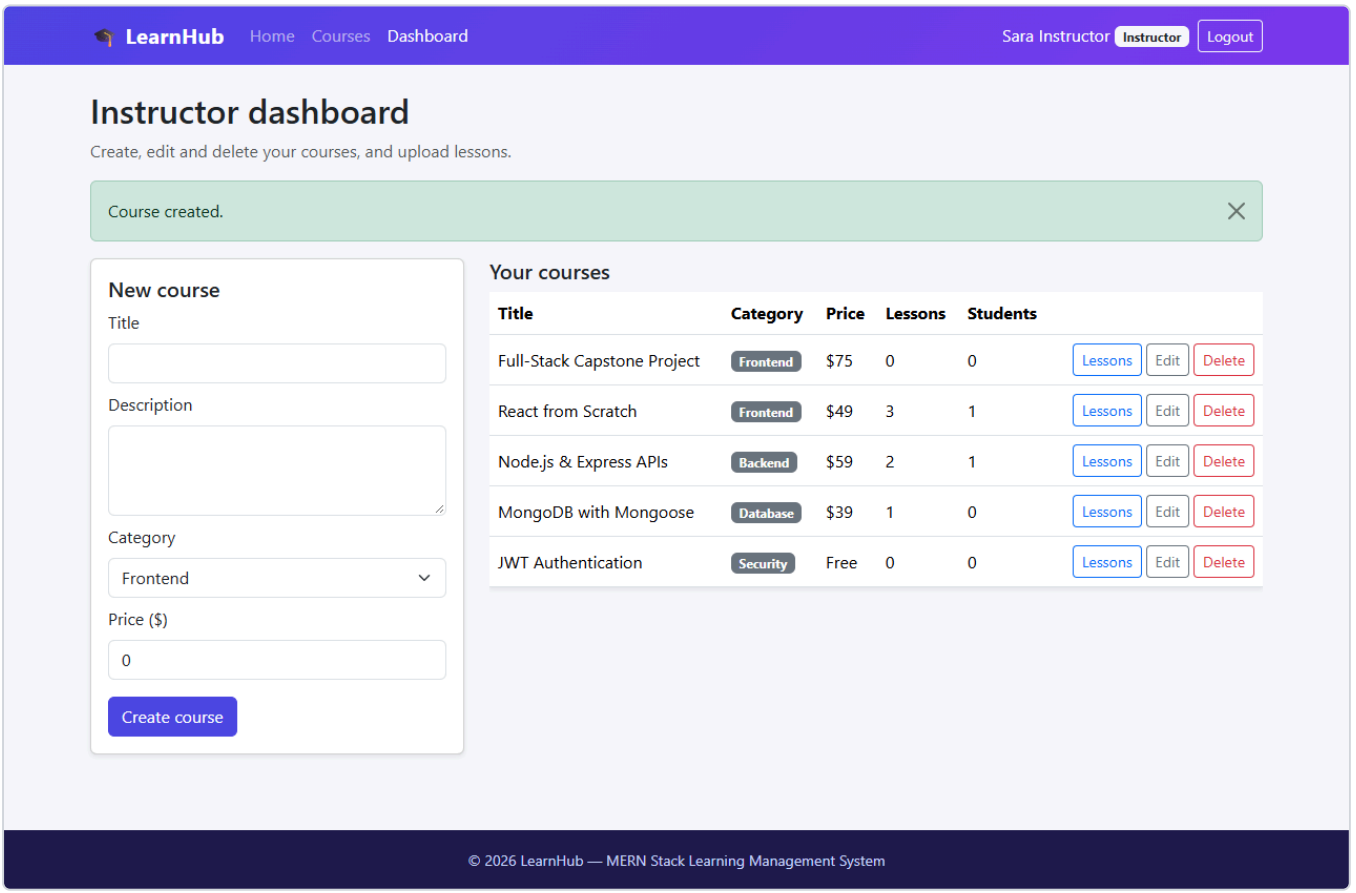
A student opening the instructor dashboard gets "Access denied"



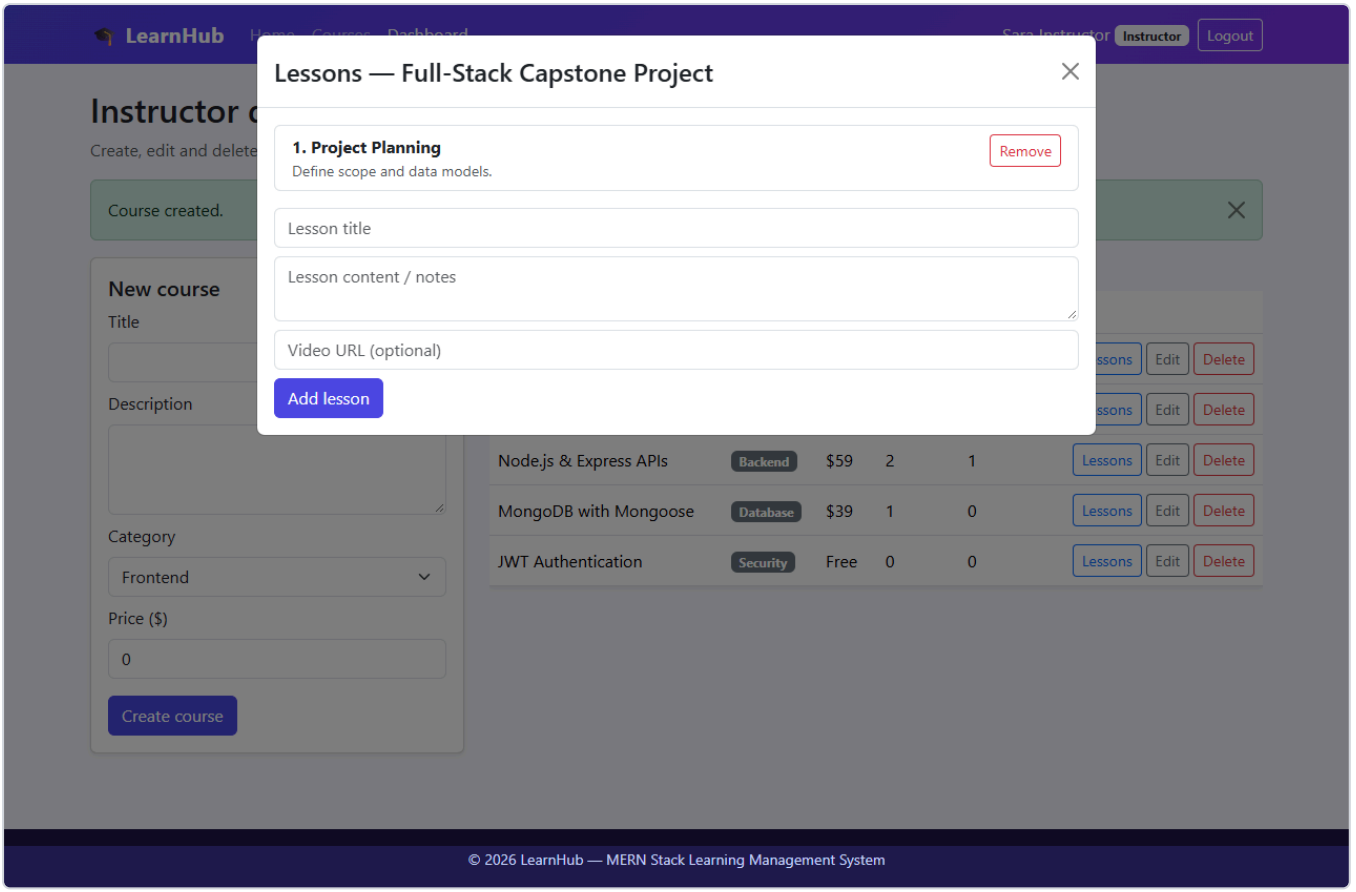
Student dashboard — enrolled courses with progress



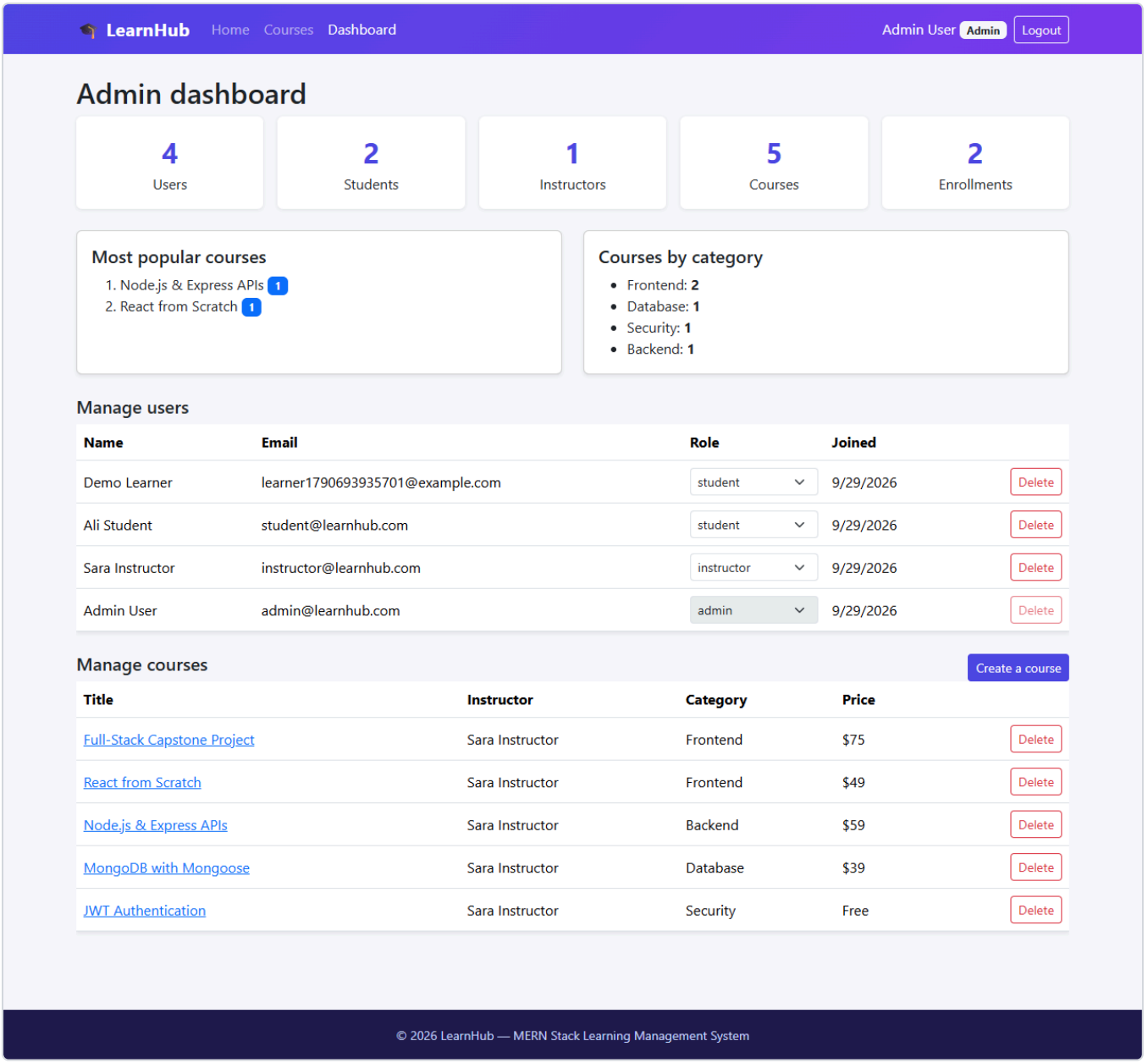
Instructor dashboard — course form and course table



Instructor created a course and the list updated from the backend



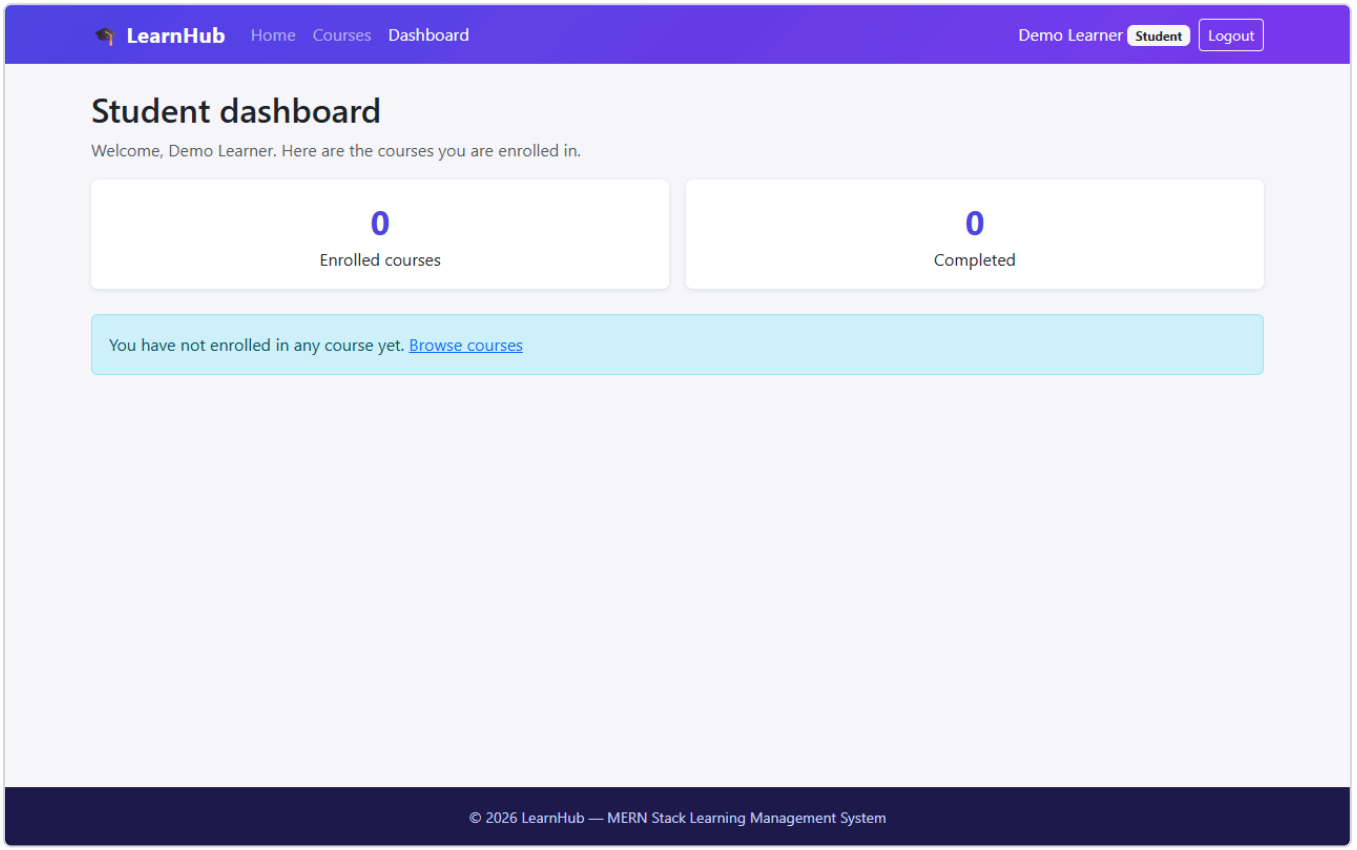
Instructor uploading lessons to a course



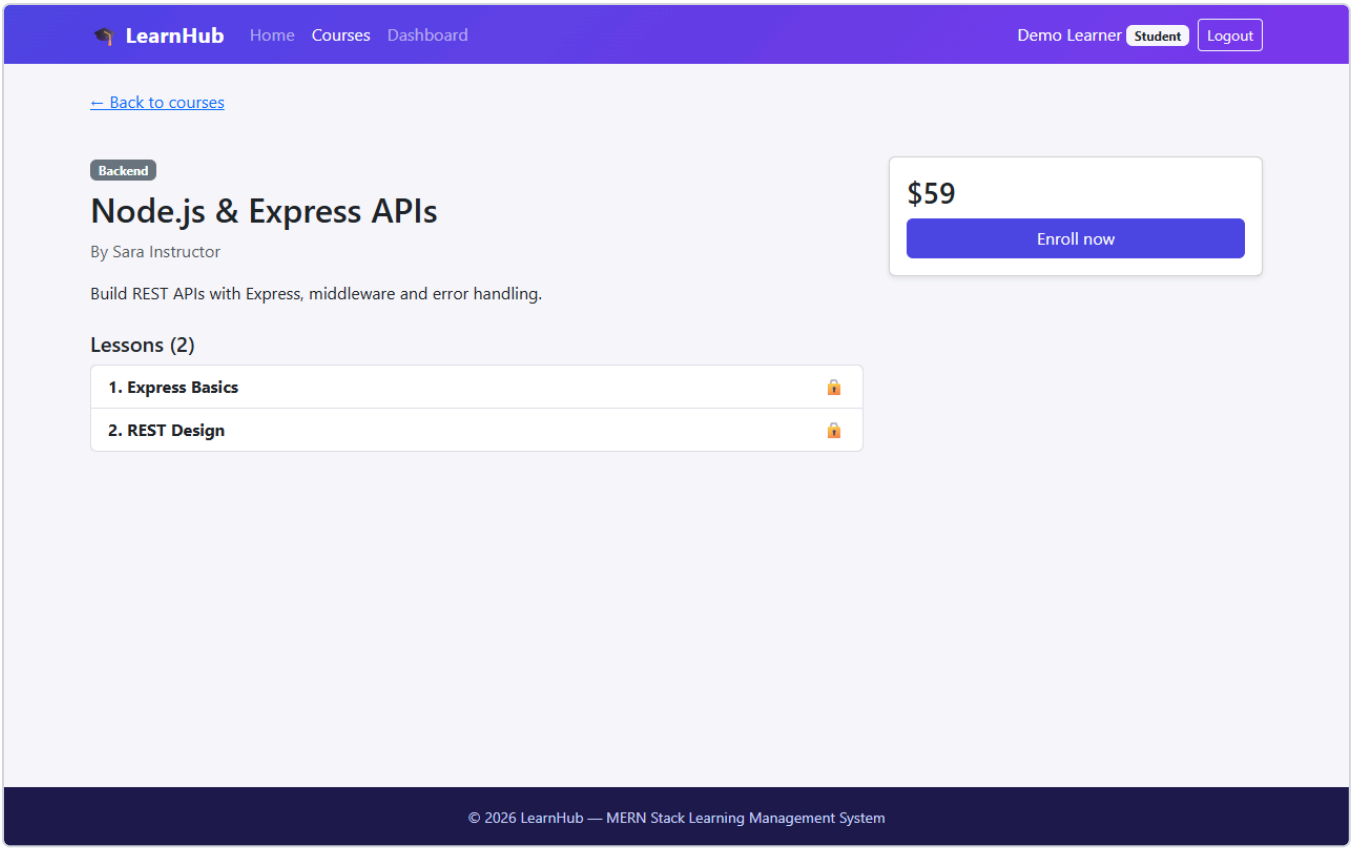
Admin dashboard — analytics, user role management, course management

9. Question 2 — Enrollment End-to-End

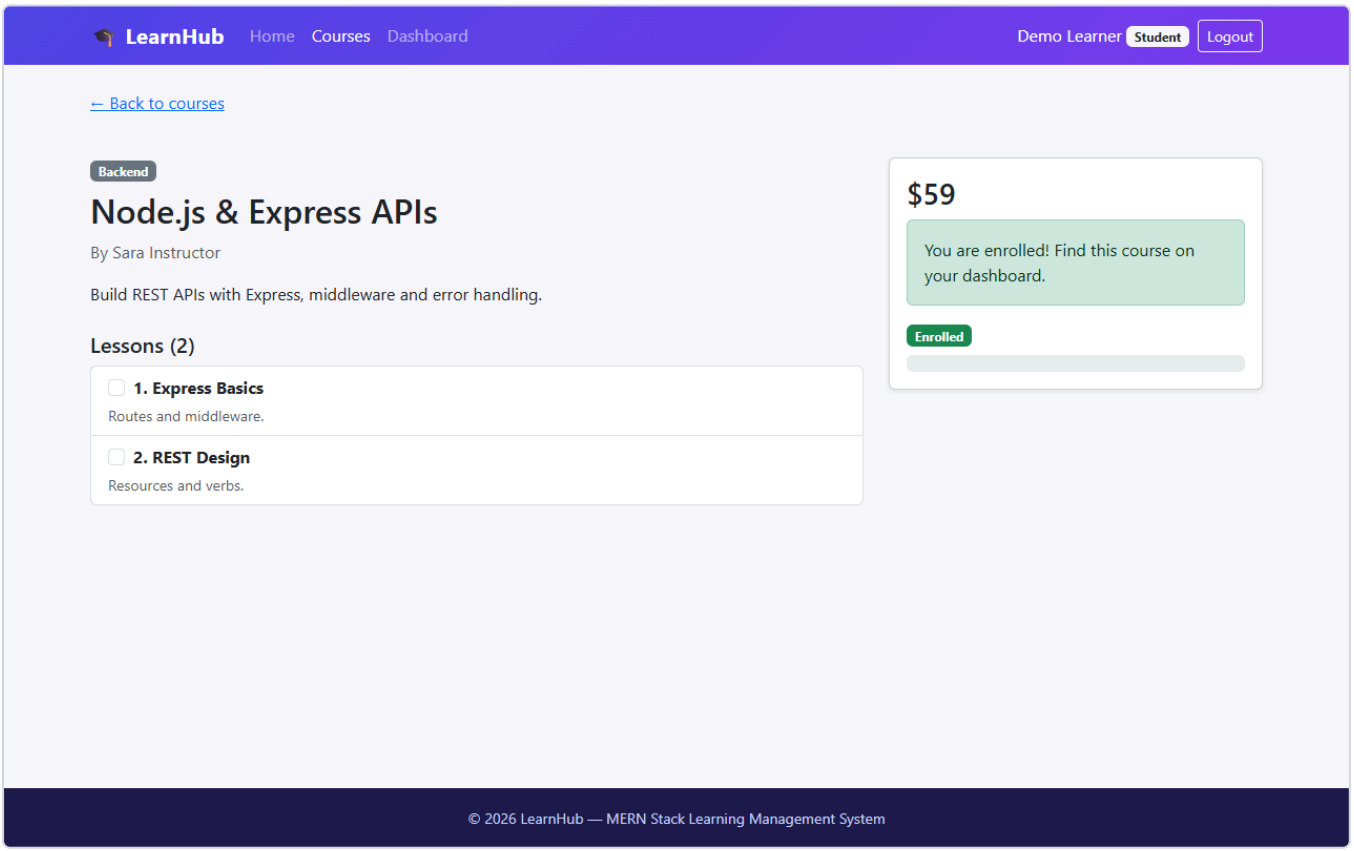
A brand-new student registers, enrolls in a course and sees it on the dashboard; completing a lesson updates progress.



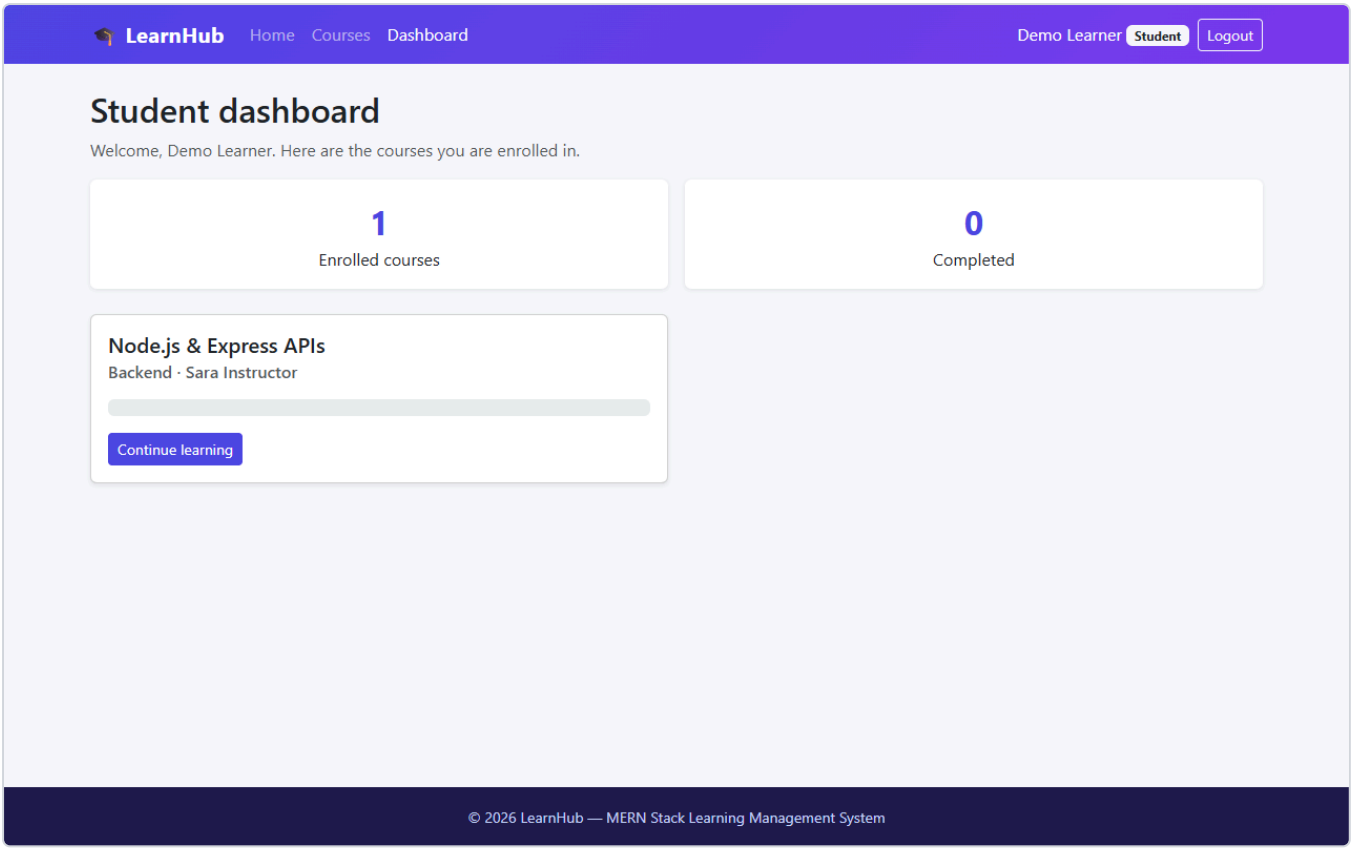
Step 1 — newly registered student has no enrollments



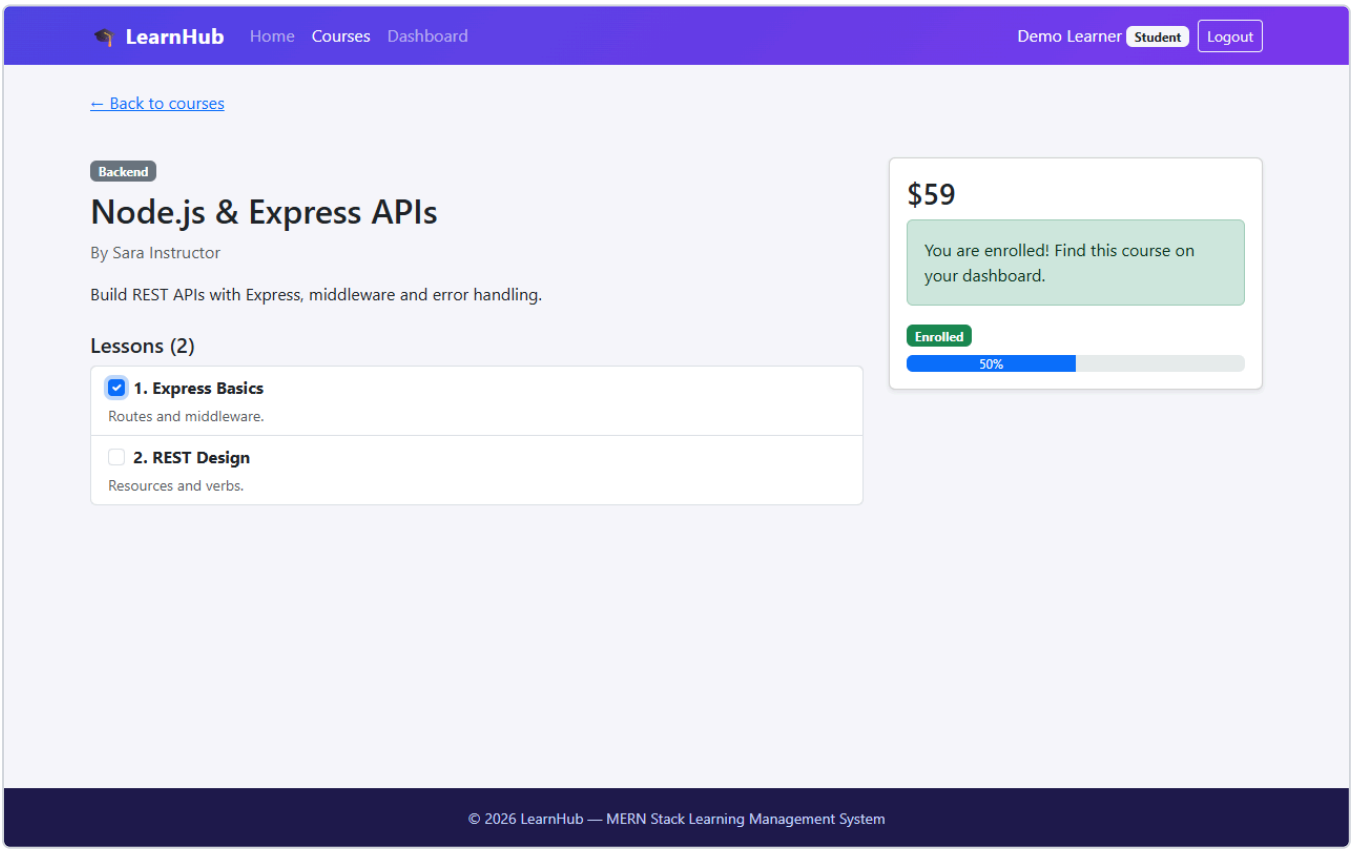
Step 2 — student opens a course and sees "Enroll now"



Step 3 — POST /api/enroll succeeded



Step 4 — course appears on the Student dashboard (GET /api/my-courses)



Step 5 — completing a lesson updates progress

10. Testing and Deployment Readiness

An automated end-to-end API test (`backend/tests/api.test.js`) covers registration, login, validation errors, protected routes, role restrictions, course CRUD, enrollment and admin actions.

```
● ● ●  npm test

Auth
  PASS  register rejects invalid email (400)
  PASS  register rejects short password (400)
  PASS  register cannot self-assign admin (400)
  PASS  register student (201 + token)
  PASS  duplicate email rejected (409)
  PASS  register instructor (201)
  PASS  login wrong password (401)
  PASS  login ok (200 + token)
  PASS  seeded admin can log in
Protected routes
  PASS  no token -> 401
  PASS  bad token -> 401
  PASS  student cannot create course (403)
  PASS  instructor cannot view analytics (403)
Course CRUD
  PASS  invalid course rejected (400)
  PASS  instructor creates course (201)
  PASS  GET /courses lists new course
  PASS  GET /courses/:id (200)
  PASS  invalid id -> 400
  PASS  owner updates course
  PASS  instructor adds lesson (201)
Enrollment
  PASS  student enrolls (201)
  PASS  double enrollment rejected (409)
  PASS  instructor cannot enroll (403)
  PASS  GET /my-courses shows enrollment
  PASS  completing lesson sets progress 100
Admin
  PASS  admin analytics (200)
  PASS  admin lists users
  PASS  admin changes role
Cleanup
  PASS  owner deletes course (200)
  PASS  deleted course -> 404
  PASS  admin deletes user
  PASS  admin deletes instructor

32 passed, 0 failed
```

Deployment steps

1. Create a MongoDB Atlas cluster and copy its connection string.
2. `cd frontend && npm run build` — creates `frontend/dist`.
3. Deploy the repository as a Node web service (e.g. Render): build command `npm --prefix frontend install && npm --prefix frontend run build && npm --prefix backend install`, start command `node backend/server.js`.
4. Set environment variables: `MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`.

Express serves the built React app and the API from one origin, so no extra CORS or proxy configuration is needed in production. The full application was verified end-to-end running locally at `http://localhost:5000` against a real MongoDB service, as shown above.

11. Conclusion

LearnHub meets all requirements of Questions 1 and 2: React pages with routing, Express REST APIs with MongoDB, JWT authentication with bcrypt, role-based authorization on both backend and frontend, an enrollment feature, clear error feedback, and deployment readiness.